

FeatureLiftBench: A Benchmark for Behavior-Preserving Feature Lifting Across Software Boundaries

ANONYMOUS AUTHOR(S)

Software reuse is fundamental to modern software development, and coding agents are increasingly used to locate, adapt, and reuse functionality from existing repositories. Yet existing coding-agent benchmarks largely focus on modifying repositories or generating functionality from specifications, leaving the reconstruction of existing capabilities across independent package boundaries underexplored. We introduce behavior-preserving feature lifting, where an agent uses an intact source repository as implementation evidence to construct an independent package that preserves specified behavioral obligations while allowing explicitly declared destination adaptations, without runtime access to the original repository. We instantiate this setting in FEATURELIFTBENCH, a benchmark of 150 Python tasks from 126 real repositories, and evaluate six OpenHands model-harness configurations. Repository evidence substantially improves functional success, yet failures often persist even after agents inspect source code associated with the requested capability. Our analysis identifies recurring mismatches in assumptions, supporting representations, and destination-specific adaptations. These results indicate that feature lifting is not merely a localization or copying problem: successful reuse requires translating source evidence into behavior that remains correct across a new software boundary.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; *Software verification and validation*.

Additional Key Words and Phrases: coding agents, software reuse, feature extraction, behavioral preservation, benchmark

ACM Reference Format:

Anonymous Author(s). 2026. FeatureLiftBench: A Benchmark for Behavior-Preserving Feature Lifting Across Software Boundaries. 1, 1 (September 2026), 20 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Software reuse involves selecting, adapting, and integrating existing artifacts [Krueger 1992]. Moving a capability across project boundaries is one such reuse scenario: a developer may need a parser from a larger tool, configuration handling from a framework, or a signal registry for a new library. An implementation that works within its source repository may depend on supporting modules, shared state, framework mechanisms, configuration, or resources that do not automatically cross a new package boundary. Reuse therefore requires more than locating the apparent entrypoint. This raises a software engineering question: *which behavioral obligations remain unmet when an existing capability is reconstructed across a new package boundary, even when its original implementation is available?* We study this question through coding agents that construct independent packages from intact repositories and explicit behavioral contracts.

We formulate this task as *behavior-preserving feature lifting*, or *feature lifting* for short. Given a version-pinned source repository and a public behavioral contract, an agent constructs a new package that is evaluated without runtime access to the source repository. The agent may extract, adapt, or reimplement code; correctness is determined by the observable behavior required at the

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/9-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

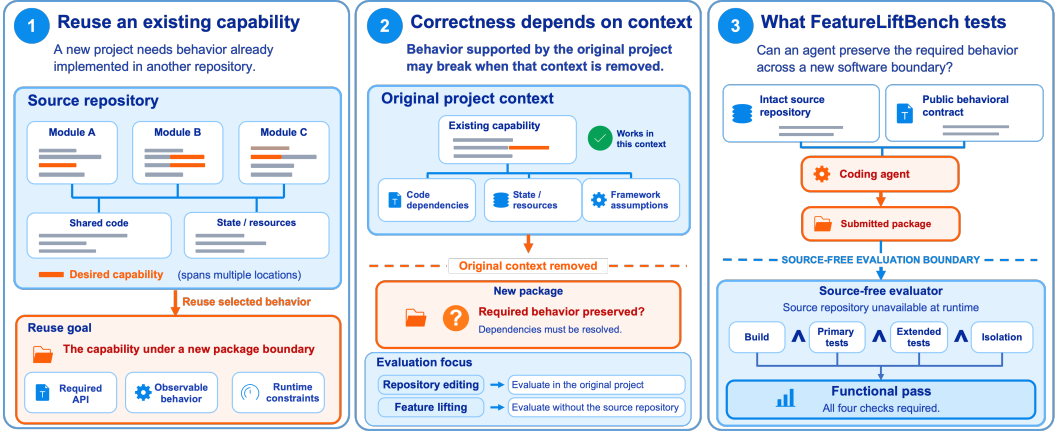


Fig. 1. Motivation and task overview of FEATURELIFTBENCH. An agent receives the intact source repository and a public behavioral contract, constructs a new package, and submits only that package to source-free evaluation. The repository structure and highlighted locations are illustrative; agents receive no benchmark-provided source-location hints.

new package boundary while allowing explicitly declared destination adaptations. Source-free evaluation requires the agent to reconstruct the support needed for that behavior, while the source repository remains available as implementation evidence during construction. This setting exposes whether the requested capability can operate across the new boundary without relying on the original project context. Figure 1 illustrates this setting.

This setting differs from several closely related repository-level coding settings. SWE-bench and SWE-Bench Pro ask agents to modify an existing project in response to an issue [Deng et al. 2025; Jimenez et al. 2024], while FeatBench and FeatureBench study feature implementation in an existing, partially removed, or specification-driven codebase [Chen et al. 2026a; Zhou et al. 2026]. Classical software transplantation moves donor functionality across project boundaries, but integrates it into a supplied host program rather than producing a standalone package [Barr et al. 2015a].

Table 1 summarizes these distinctions. Repository repair and in-repository feature development retain the original project as the execution context. Specification-driven standalone construction, as in FeatureBench L2 and RepoZero [Zhang et al. 2026], can produce an independent artifact but does not expose the intact source implementation as construction evidence. Software transplantation has access to donor source and crosses a project boundary, but targets integration into a supplied host. Feature lifting combines these otherwise separate properties: the intact source implementation remains available as evidence during construction, while the agent must reconstruct the requested capability across a new software boundary into an independent package that satisfies an explicit destination specification under source-free evaluation.

To study this problem, we introduce FEATURELIFTBENCH, a benchmark of 150 Python tasks constructed from capabilities in 126 real repositories. Each task combines a public behavioral contract with protected behavioral evaluation and a source-free execution boundary. We preserve task-level contract and test mappings, check their structural consistency, and report repeated reference execution. The available records do not establish complete per-task human semantic review.

Table 1. Positioning FEATURELIFTBENCH among closely related repository-level coding settings. Columns distinguish the source visible to the agent, public task information, destination boundary, required output, and evaluation context. “Source oracle” means the original repository supplies expected behavior for testing; it need not be visible to the agent. The comparison does not rank task difficulty.

Setting	Source to agent	Public task information	Destination	Output	Evaluation context
SWE-bench	Full repo	Issue	Same project	Patch	Patched project
SWE-bench Pro	Full repo	Issue + requirements / interface	Same project	Patch	Patched project
FeatBench	Full repo	Natural-language requirements	Same project	Patch	Patched project
FeatureBench L1	Partial repo	Task + interface	Same project	Callable feature	Patched project
FeatureBench L2	No repo	Task + interface	Standalone code	Callable library	Source restored for tests
RepoZero	No repo	API specifications + examples	New repository / language	New repository	Source oracle
Software transplantation	Donor + host	Annotations + tests	Donor to host	Integrated graft	Host program
FEATURELIFTBENCH	Full repo	Behavioral contract	Donor to new package	Independent package	Package only; donor absent

We evaluate multiple coding-agent configurations on the same benchmark. Functional success varies substantially across configurations, with pass rates ranging from 24.0% to 76.7%. A paired ablation shows that access to the intact repository materially improves reconstruction, confirming that source evidence is useful. However, source access alone is insufficient: 79.5% of Primary/Extended-first failures contain confirmed reads of endpoint-associated source-file content. Qualitative analysis further shows that failures can persist when supporting assumptions, representations, or destination-specific adaptations are not reconstructed correctly. Together, these findings suggest that feature lifting is not merely a localization or copying problem, but a problem of translating source evidence into behavior that remains correct across a new software boundary.

This work makes three contributions:

- **A task formulation for behavior-preserving feature lifting.** We define a repository-level reuse setting in which agents reconstruct an existing capability across a new software boundary, with intact implementation evidence available during construction but no runtime dependence on the source repository.
- **FeatureLiftBench, an executable benchmark for feature lifting.** We construct and validate 150 tasks from 126 real Python repositories, with explicit behavioral contracts, protected tests, source-free evaluation, and executable reference packages.
- **A process-level diagnosis of feature-lifting failures.** Trace and artifact analyses show that failures often persist after source exposure, with recurring mismatches in assumptions, supporting representations, and destination-specific adaptations.

2 The FeatureLiftBench Benchmark

2.1 Task Formulation and Running Example

A FEATURELIFTBENCH task pairs a pinned source repository R with a public behavioral contract S . A model backend M acting through an agent harness H produces a submitted artifact A :

$$A = \text{Run}(M, H, R, S). \quad (1)$$

The benchmark authors declare *source entrypoints* in task metadata as fully qualified upstream symbols, such as functions, classes, methods, or attributes, that anchor the requested capability (for example, `platformdirs.user_data_dir`). They identify candidate implementation locations, not the destination API or a complete list of supporting code. We use them offline to map source-file exposure and to prepare the mechanical extraction baseline. In the main comparison and paired ablation, these entrypoint fields are removed from agent-visible task materials; agents must locate the implementation themselves. The initial workspace contains the rendered public task, redacted metadata, a dependency lock, and an empty submission directory, plus the pinned repository under `repo/` when source access is enabled. The protected evaluator is separate from this construction workspace.

A feature-lifting contract defines the behavior required at the destination boundary [Meyer 1992]. These requirements may include retained upstream obligations, explicitly declared adaptations, and destination-specific obligations. Behavior preservation therefore concerns the obligations retained across the new boundary, while declared adaptations specify permitted or required differences from the upstream behavior. The contract makes the API surface, relevant defaults, exceptions, state changes, ordering constraints, and exclusions explicit. The agent may inspect the complete pinned repository, including upstream tests and documentation, but must construct the result in a designated submission directory. Benchmark tests, reference implementations, test-to-contract mappings, and benchmark-provided source-location hints remain hidden.

The required output is an independently executable package under the benchmark environment. The package may extract, adapt, or reimplement upstream code, but it must execute without runtime access to the source repository. Multiple structurally different implementations may therefore satisfy the same task.

Functional success is defined as

$$F(A) = B(A) \wedge T_{\text{primary}}(A) \wedge T_{\text{extended}}(A) \wedge I(A), \quad (2)$$

where B checks that the submitted package can be loaded under the frozen evaluation environment, T_{primary} and T_{extended} exercise protected behavioral obligations, and I checks source independence. All four conditions must hold.

Running example: a signal registry. The Blinker task asks the agent to reconstruct `Signal` and `Namespace` under a new package boundary. Its contract requires behavior such as sender-specific dispatch, weak-receiver cleanup, stable namespace identity, and scoped connection management, while explicitly excluding asynchronous dispatch and the upstream global named-signal singleton. Thus, an implementation whose central connect and send paths work can still fail if supporting state or lifetime behavior is lost. The agent may reuse or reimplement the relevant upstream logic, but the resulting package must satisfy the contract without importing or reading the original blinker source at runtime.

2.2 Task Construction

The frozen 150-task membership identifies 126 repositories and 132 pinned source snapshots. Each retained task links a public contract, generated task statement, protected test nodes, and a source-independent reference package. Figure 2 gives a concrete example of these links. The anonymous construction-evidence bundle contains the membership and source digests, per-task contract-test mappings and test files, recorded review fields, and reference replay results.

The public specification declares required APIs, behavior, adaptations, and exclusions. The generated task statement exposes those requirements to the agent; Primary and Extended tests remain hidden. Per-task records map test nodes to public clauses and required APIs, and the frozen

Blinker asset	Recorded link or check
Pinned source	Repository snapshot identity and digests in the frozen membership.
Public contract	Signal, Namespace, and ANY; dispatch, weak receivers, namespace identity, and source independence.
Test mapping	Sender dispatch and weak-receiver test nodes link to public behavior clauses; the stored links are broader than individual test assertions.
Feasible reference	Three recorded source-free passes for this task.

Fig. 2. Traceable assets for the Blinker signal-registry task. The example shows links preserved in the evidence bundle; structural mapping checks do not certify semantic precision of every test-to-clause link.

manifest binds each task to a source snapshot. A feasible reference package is retained for each task. These records describe the retained benchmark, but do not reconstruct the original candidate-selection decisions: a complete selection and rejection ledger for the 150 tasks was not located. We therefore report no historical candidate counts or exclusion criteria as executed facts.

In the Blinker example, a sender-dispatch test exercises the public dispatch and sender-filtering obligations, while a separate weak-receiver test exercises receiver lifetime. The stored mapping connects each behavior test to all four behavior clauses, so it is coarser than these test bodies. The evidence bundle preserves both the original mapping and a dated inspection of this example. The same frozen 150-task membership is used for every configuration in the main comparison.

2.3 Benchmark Validation

Validation must address whether evaluator judgments capture the intended behavior, an instance of the test-oracle problem [Barr et al. 2015b]. The available records support structural consistency and executable feasibility checks. Semantic alignment requires reading the contract and test assertions together; its human-review coverage is not fully documented.

Structural consistency. Our dated retrospective audit checks generated task statements, hashes, declared API and behavior coverage, test node IDs, static test API use, and agreement between companion mapping records. All 150 retained task packages pass these checks and resolve to 132 pinned source snapshots. A structural pass does not establish that every mapped test assertion fairly implements its public clause.

Semantic alignment. Task metadata records AI-assisted review entries for all 150 tasks; separate contract records label 143 as AI-assisted reviewed and seven as maintainer reviewed. We found no complete per-task human-review ledger with reviewer, date, contract-assertion judgment, and disposition. We preserve these fields without upgrading them to a claim of complete human review. The dated Blinker inspection above illustrates why structural coverage alone is insufficient; full semantic re-review requires recorded human judgments across the collection.

Executable feasibility. Saved records show three successful source-free reference replays per task: 450/450 executions (150 tasks $\times$ 3 runs). These records establish a feasible solution for each retained task under the recorded evaluator. They do not establish semantic completeness, and this evidence assembly does not rerun the references.

2.4 Dataset Composition

We characterize the 150 tasks along three complementary dimensions: *functional family* describes what capability is lifted, *lift type* describes how the requested capability relates to its upstream implementation, and *entanglement* describes which forms of project dependency must be handled

when crossing the new package boundary. These annotations describe benchmark composition and do not affect scoring.

Functional coverage. The benchmark spans ten functional families across 126 repositories and 132 pinned source snapshots. The largest families are parsing/tokenizing/decoding (31 tasks), registry/plugin dispatch (22), serialization/formatting/rendering (19), configuration resolution/discovery (18), and validation/normalization/construction (15). The remaining tasks cover workflow and session orchestration, resource and metadata loading, algorithms and data structures, protocol state transitions, and cache/retry policies.

Lift types. Each task receives one of three labels according to the relation between the requested capability and its upstream implementation. *Direct* tasks preserve an identifiable upstream capability while extracting it from its original project context. *Adapted* tasks retain an identifiable capability but require a declared change to its interface or observable behavior. *Composite* tasks combine multiple upstream capabilities or reconstruct a workflow without a single corresponding upstream API. The benchmark contains 56 Direct, 76 Adapted, and 18 Composite tasks. These labels characterize the requested transformation rather than the implementation strategy used by an agent.

Entanglement mechanisms. Tasks may additionally carry multiple labels describing the dependencies that must be accounted for during lifting. We group the underlying annotations into four mechanisms: *code dependencies*, covering internal helpers, transitive imports, and third-party interfaces; *data and state*, covering shared models, parser state, and registries; *framework mechanisms*, covering lifecycle, dispatch, and dynamic discovery; and *environment and resources*, covering configuration, path rules, and packaged resources. These mechanisms are non-exclusive and describe where a capability is coupled to its source project rather than independent difficulty factors.

Figure 3 summarizes the resulting composition. Panel (a) shows the ten functional families partitioned by lift type, while panel (b) shows the prevalence of the four entanglement mechanisms overall and within each lift type.

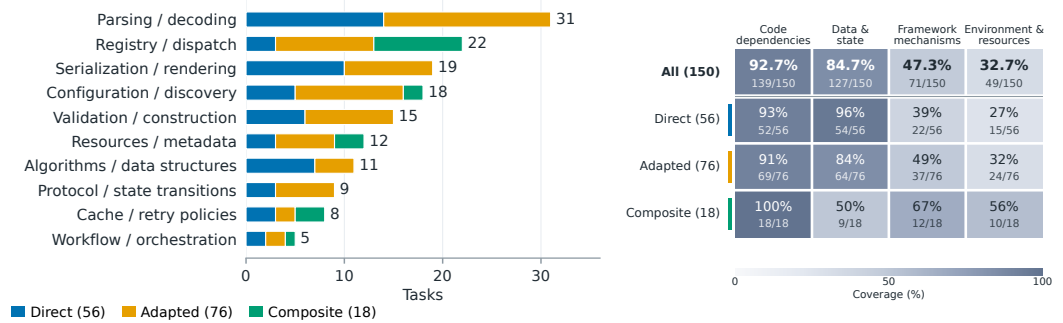


Fig. 3. Composition of the 150-task benchmark. (a) Ten functional families partitioned by lift type: Direct (56), Adapted (76), and Composite (18). (b) Coverage of four non-exclusive entanglement mechanisms overall and within each lift type; cells report percentages and task counts. These annotations characterize benchmark composition rather than predefined difficulty.

3 Evaluation Protocol and Experimental Setup

3.1 Agent Configurations

We evaluate DeepSeek V4 Pro (deepseek-v4-pro-0813), DeepSeek V4 Flash (deepseek-v4-flash-0731), GPT-5.6 Luna, GLM-5.3-Flash, Qwen3.6-35B-A3B, and GPT-OSS 120B, abbreviated as Pro, Flash, Luna, GLM, Qwen, and OSS. All configurations use OpenHands CLI 1.16.0 with native tool calling [Wang et al. 2025] and follow the information boundary defined in Section 2.

The reported execution settings for the main comparison and paired ablation include a 3,600-second timeout and a 131,072-token context window with 8,192 tokens reserved for output. A later author clarification reports a 150-step maximum, whereas the locally available Pro and Flash main-run metadata records 120. We cannot establish a common effective step cap across all retained runs from the current archive. In the main comparison, Pro and Flash use token-based context condensation; the remaining configurations use the default OpenHands context handling. Work on agent-computer interfaces and harness effects motivates reporting these execution choices as part of the evaluated system [Yang et al. 2024; Yao et al. 2026]. We therefore treat each model-harness configuration as the experimental unit rather than attributing observed differences to the base model alone.

3.2 Functional Evaluation and Artifact Metrics

Only the collected submission enters the evaluator environment; the source repository is unavailable at runtime, networking is disabled, and dependencies are fixed. Functional success follows Equation 2. The **Build** gate checks that the submitted package can be loaded under the frozen evaluator environment; **Primary** and **Extended** use withheld tests to exercise behavioral obligations specified by the same public contract; and **Isolation** checks prohibited source dependencies and protected-path access. Protected paths are evaluator-private locations such as /workspace/tasks, /workspace/benchmark/sources, and /workspace/benchmark/references, as well as named reference or registry files; submitted code cannot use them to recover hidden evaluation material. Task contracts separately prohibit runtime access to the original repo/ snapshot, which is absent from the evaluator environment.

Successful-artifact characteristics. For a functionally passing artifact A and its frozen feasible reference A_{ref} , we define Reference-Relative Extraction Size as

$$\text{RRES}(A) = \frac{\text{PythonLOC}(A)}{\text{PythonLOC}(A_{\text{ref}})}. \quad (3)$$

RRES characterizes artifact size relative to one feasible reference; it does not measure minimality.

We additionally report *Copy*, the fraction of submitted Python source matched to the source repository in normalized runs of at least three non-empty, non-comment lines. Copy measures syntactic overlap rather than semantic equivalence, a distinction established in code-clone research [Roy et al. 2009], and can miss rewritten equivalents. Both RRES and Copy are reported only after functional success and are not incorporated into the functional score. RRES and Copy capture complementary aspects of implementation footprint: RRES measures implementation size relative to a feasible reference, while Copy measures direct source-code overlap with the source repository. FEATURELIFTBENCH therefore separates functional correctness from implementation footprint: a lift succeeds by satisfying destination behavior and source independence, while RRES and Copy characterize how compactly and how directly the source implementation is reused.

Execution effort. Table 2 reports logged assistant-event counts and total token usage for all assigned tasks. The event count is taken from saved usage records; it is not interchangeable with

the runtime’s configured iteration limit. Total tokens are computed as input, including cached input, plus output tokens for each run.

3.3 Comparison Scope and Statistical Analysis

The main comparison contains one selected outcome for each of six configurations on each of the same 150 tasks, yielding 900 model–task outcomes. The saved Flash records include 24 second attempts after first attempts raised a suite exception before agent execution and produced no submission; the second attempts supply the reported outcomes for those cells (12 passing and 12 failing). The available records do not establish that every other cell had exactly one task-level attempt. All assigned tasks remain in the denominator, including runs that produce no usable submission. Functional correctness is determined by the frozen evaluator rather than by the agent runner’s terminal status.

We treat the resulting pass rates as observed performance of the evaluated model–harness configurations. Because each configuration contributes one selected outcome per task, the main comparison does not estimate run-to-run stochastic variance.

The repository-evidence ablation compares Full Source and Contract Only on the same tasks using paired pass-rate differences, paired bootstrap confidence intervals, and exact conditional McNemar tests [Fagerland et al. 2013], with Holm correction across configurations [Holm 1979]. Intervals use 100,000 paired task-bootstrap resamples and describe variation across task pairs rather than repeated agent runs.

We compare successful-artifact footprints using task fixed effects to account for differing successful task sets. We retain tasks with successful artifacts from at least two configurations and fit:

$$\begin{aligned}\log_2(\text{RRES}_{t,m}) &= \alpha_t^{\text{RRES}} + \beta_m^{\text{RRES}} + \epsilon_{t,m}^{\text{RRES}}, \\ \text{Copy}_{t,m} &= \alpha_t^{\text{Copy}} + \beta_m^{\text{Copy}} + \epsilon_{t,m}^{\text{Copy}},\end{aligned}\tag{4}$$

with $\sum_m \beta_m^{\text{RRES}} = \sum_m \beta_m^{\text{Copy}} = 0$. Here t indexes tasks and m indexes configurations. Ordinary least squares gives each successful artifact equal weight. We report the task-adjusted RRES ratio $2\beta_m^{\text{RRES}}$ and the Copy difference $100\beta_m^{\text{Copy}}$ in percentage points. Their reference centers, $1\times$ and 0 pp, represent the sum-to-zero center of configuration effects; the RRES ratio is not an adjusted median or a raw size ratio to the reference implementation.

Pointwise 95% percentile intervals use task-cluster bootstrap resampling [Field and Welsh 2007]. Sensitivity checks using task-equal weighting and repository-level resampling preserve the qualitative results. Resampling details are provided in the replication materials. These intervals describe variation across tasks or repositories, not repeated agent runs. The analysis is descriptive and conditional on success; it does not estimate causal configuration effects or a global quality ranking.

3.4 Paired Repository-Evidence Ablation

The paired ablation removes repository evidence while keeping the requested behavior fixed.

We randomly sample a fixed 40-task subset, stratified by lift type, containing 15 Direct, 20 Adapted, and 5 Composite tasks from 38 repositories. The subset was frozen before inspecting ablation outcomes. GPT-5.6 Luna, DeepSeek V4 Pro, and Qwen3.6-35B-A3B are each evaluated under two conditions on the same tasks, yielding 240 retained outcomes. These are new paired runs rather than reused outcomes from the main comparison.

In the *Full Source* condition, the agent receives the complete pinned repository together with the public behavioral contract. In *Contract Only*, the repository is omitted while the same contract and permitted dependency information are retained. This information comprises the task’s allowlisted third-party packages in redacted metadata and their pinned versions in requirements.lock; the

lock lists no packages for standard-library-only tasks. Retrieving the original project from the network, installed packages, or caches is prohibited. Benchmark tests, reference implementations, and source-location hints remain hidden in both conditions. The intervention therefore removes repository evidence as a whole, including source code, upstream tests, documentation, and packaged resources, rather than isolating source-code text alone.

Within each configuration, the paired conditions use the same model-harness settings and source-free evaluator. The Pro configuration in this ablation does not apply context compression. Missing submissions are counted as functional failures; paired statistical methods are specified above.

3.5 Trace-Based Source-Exposure Measurement

To distinguish source availability from observable source use, we analyze the 900 saved trajectories from the main comparison. For each task, we statically map its declared source entrypoints to files in the pinned repository. This resolves 607 of 641 declared entrypoints; at least one entrypoint is resolved for 145 of the 150 tasks.

We call an event a *confirmed source read* when a successful tool read returns content matching at least two adjacent normalized source lines from a mapped file. Filename mentions, unsuccessful reads, and traceback text do not qualify. Unresolved mappings remain unconfirmed rather than being treated as absence of source inspection. This measure therefore captures observable exposure to entrypoint-associated source-file content, not whether the agent located the complete implementation, followed all supporting dependencies, or understood the returned code. We use it as a diagnostic of whether behavioral failures can persist despite confirmed reads of entrypoint-associated source-file content.

Qualitative analysis of source-exposed Primary/Extended-first failures. We inspect a fixed, purposively diverse sample of 40 source-exposed Primary/Extended-first failures, with one case per task. The selected sample contains seven cases each for Pro, Flash, Luna, and GLM, and six each for Qwen and OSS. Selection also covers functional families, lift types, and prior review status, with deterministic hash-based tie-breaking. The sample spans all six configurations, ten functional families, and all three lift types.

Two authors independently reviewed the public contract, pinned upstream implementation, submitted artifact, and evaluator outcome for each case, and reconciled disagreements to derive a qualitative coding framework. We synthesize the reviewed evidence into nonexclusive themes that relate observable obligations to supporting software relationships and distinguish upstream preservation from destination adaptation. Cases not captured by recurring themes are retained as other or case-specific observations; the analysis identifies qualitative patterns rather than their prevalence. The replication materials preserve the complete case-to-theme mapping and review history. Three illustrative cases are paired with successful artifacts under the same contracts.

3.6 Retrospective Execution-Effort Analysis

We recover token usage from provider audit logs and persisted per-response metrics, including agent and context-compression calls. Shared parent/child records are deduplicated by response ID. Total token usage is computed as the sum of input and output tokens across all model calls in each run.

For trajectory-level analysis, we evaluate reconstructed artifacts at saved tool-completion boundaries and identify the first *observed passing checkpoint*. With cumulative tokens T_r^{obs} at this checkpoint and run total T_r , the post-sufficiency fraction is $\text{PSF}_r = (T_r - T_r^{\text{obs}})/T_r$. Token timing requires one-to-one response-ID alignment with the event stream, including compression responses, and

Table 2. Main comparison on the common 150-task benchmark. Pass@1, logged assistant events, and Tokens are computed over all assigned outcomes, while RRES and Copy summarize only functionally successful artifacts. RRES measures reference-relative extraction size, and Copy measures normalized source-code overlap.

Configuration	Correctness	Artifact footprint		Assistant events		Tokens (10 ⁴)	
	Pass@1 <i>n</i> (%)	RRES Median [Q1, Q3]	Copy Median [Q1, Q3]	Median	P90	Median	P90
DeepSeek V4 Pro	115 (76.7)	0.99 [0.77, 1.29]	0.96 [0.72, 0.99]	41.5	80.1	242.39	635.17
DeepSeek V4 Flash	108 (72.0)	1.00 [0.89, 1.22]	0.97 [0.84, 0.99]	63.0	116.2	422.59	968.26
GPT-5.6 Luna	102 (68.0)	0.81 [0.31, 1.00]	0.16 [0.05, 0.97]	35.5	74.4	148.37	1134.94
GLM-5.3-Flash	68 (45.3)	1.01 [0.92, 1.16]	0.95 [0.81, 0.98]	85.5	127.0	701.85	1879.19
Qwen3.6-35B-A3B	63 (42.0)	0.97 [0.65, 1.23]	0.76 [0.33, 0.97]	41.5	106.6	181.86	552.10
GPT-OSS 120B	36 (24.0)	0.98 [0.29, 1.65]	0.18 [0.01, 0.53]	19.0	61.1	52.60	187.73

agreement with audit counts and timing. We separately count distinct subsequent responses recorded as actions or messages in the main agent event stream, excluding the response that produced the passing artifact. This count excludes compression responses and unobserved HTTP retries. Both measures condition on final success, evaluated checkpoint chains, and matching reconstructed and re-evaluated final submissions, but use independent available samples. Values are computed per run before aggregation. These retrospective checkpoints do not establish a stopping signal available to the agent.

4 Benchmark Results

We organize the core results around three questions about agent-mediated software reuse: **RQ1** measures functional success overall and across reuse structures; **RQ2** tests the effect of repository evidence through paired ablation; and **RQ3** examines implementation mismatches in reviewed behavioral failures with confirmed reads of endpoint-associated source-file content. Secondary analyses characterize execution after an observed passing checkpoint and the implementation footprints of successful packages.

4.1 RQ1: How Does Feature-Lifting Success Vary Across Configurations and Reuse Structures?

Table 2 reports the main comparison over the same 150 tasks for all six configurations. DeepSeek V4 Pro achieves the highest observed functional success, passing 115/150 tasks (76.7%), followed by Flash at 108/150 (72.0%) and Luna at 102/150 (68.0%). GLM, Qwen, and OSS pass 68 (45.3%), 63 (42.0%), and 36 (24.0%) tasks, respectively.

Because each model-task cell contains one retained run, the values describe observed performance of the evaluated model-harness configurations rather than run-to-run success probabilities.

Observed variation across task structures. Table 3 reports pass rates within the three lift types and four overlapping entanglement-mechanism groups. Across all six configurations, observed pass rates are lower on Adapted and Composite tasks than on Direct tasks. For these configurations, success declines when reuse requires changing or composing the upstream capability boundary rather than directly preserving an identifiable capability. For example, Pro passes 91.1% of Direct tasks, 72.4% of Adapted tasks, and 50.0% of Composite tasks. These descriptive comparisons do not isolate a causal difficulty effect: category membership co-varies with other task properties, and the Composite group contains only 18 tasks. The mechanism groups also overlap substantially: a task

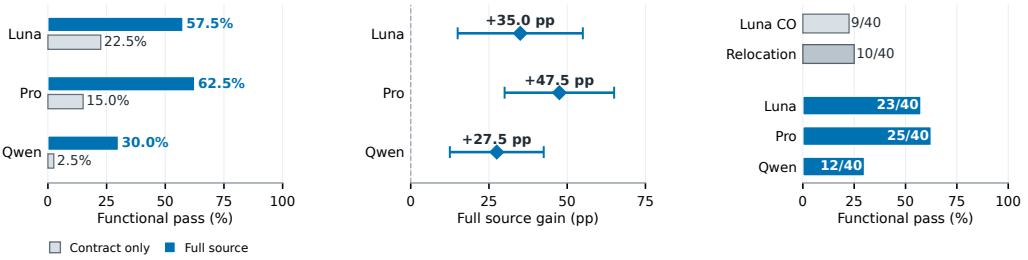


Fig. 4. Repository-evidence analyses on the same 40 tasks. (a) Functional pass rates under Contract Only and Full Source for Luna, Pro, and Qwen. (b) Full Source minus Contract Only effects with 95% paired task-bootstrap intervals. (c) Diagnostic comparison of Luna Contract Only (Luna CO), mechanical relocation, and the three Full Source agent configurations. Mechanical relocation receives frozen entryptpoint mappings and is not a controlled ablation. The 18 Pro Contract Only runs without submissions are counted as failures.

may involve both shared state and environment-dependent resources, for example. Their marginal pass rates cannot identify the effect of an individual mechanism or explain which obligation was violated in a particular failed artifact. RQ3 examines that distinction at the level of concrete contracts and implementations.

Table 3. Observed functional pass rates (%) by task structure. The first three rows are mutually exclusive lift types; the remaining rows are non-exclusive entanglement mechanisms.

Task structure	<i>n</i>	Pro	Flash	Luna	GLM	Qwen	OSS
Direct	56	91.1	89.3	80.4	71.4	55.4	28.6
Adapted	76	72.4	67.1	65.8	31.6	35.5	21.1
Composite	18	50.0	38.9	38.9	22.2	27.8	22.2
Code dependencies	139	79.9	73.4	69.1	46.0	42.4	22.3
Data and state	127	80.3	75.6	71.7	48.0	44.1	23.6
Framework mechanisms	71	78.9	73.2	66.2	40.8	43.7	16.9
Environment and resources	49	75.5	71.4	63.3	32.7	34.7	24.5

4.2 RQ2: Does Repository Evidence Improve Behavioral Reconstruction?

We investigate the role of repository evidence from two perspectives: whether access to repository evidence improves behavioral reconstruction, and whether its benefit can be reduced to mechanically relocating existing source implementations. The first analysis compares Full Source and Contract Only conditions under a paired-task setting. The second introduces an entryptpoint-guided mechanical extraction baseline to examine the limits of direct source reuse.

4.2.1 Repository evidence improves behavioral reconstruction. Figure 4 visualizes the paired effects, while Table 4 reports the exact paired statistics. Repository evidence improves Luna’s and Qwen’s functional success, with both paired differences remaining significant after Holm correction. Pro also shows a positive difference, but its Contract Only condition includes 18 runs without usable submissions. Pro’s paired effect therefore reflects both behavioral reconstruction and successful artifact delivery.

Table 4. Paired repository-evidence outcomes and statistics on 40 tasks per configuration. Full-only and Contract-only count discordant pairs. Δ is Full Source minus Contract Only (pp); intervals are 95% paired task-bootstrap intervals, and p_{Holm} gives exact McNemar p-values adjusted across the three configurations.

Configuration	Passes		Discordant pairs		Paired effect		
	Full source	Contract only	Full only	Contract only	Δ (pp)	95% CI (pp)	p_{Holm}
GPT-5.6 Luna	23/40	9/40	17	3	35.0	[15.0, 55.0]	5.15×10^{-3}
DeepSeek V4 Pro	25/40	6/40	20	1	47.5	[30.0, 65.0]	6.29×10^{-5}
Qwen3.6-35B-A3B	12/40	1/40	12	1	27.5	[12.5, 42.5]	5.15×10^{-3}

4.2.2 Mechanical relocation is insufficient. Repository evidence may benefit agents either by exposing reusable source implementations or by supporting more complex reconstruction. To distinguish these possibilities, we introduce an entryptpoint-guided mechanical extraction baseline (DSE). DSE receives frozen entryptpoint mappings and performs only predefined relocation operations, including source collection, package restructuring, import rewriting, and fixed alias mappings. It does not perform semantic modification, test-driven repair, or task-specific adaptation. Because DSE receives explicit source-location information unavailable to the evaluated agents, it serves as a diagnostic baseline rather than a controlled model ablation.

Figure 4(c) shows the DSE results on the same 40-task diagnostic subset. DSE successfully lifts 10/40 tasks, compared with 23/40, 25/40, and 12/40 for the Luna, Pro, and Qwen Full Source agent configurations, respectively. DSE produces artifacts with a median static source overlap of 99.14% across all 40 generated submissions. It uses the same normalized-line matching rule as Copy, but covers failures as well as successes, whereas Table 2 reports Copy only for successful artifacts. This diagnostic comparison shows that the specified mechanical relocation procedure does not reproduce the functional success of these Full Source agents on this subset; it does not isolate which reconstruction steps account for the gap.

Among the 30 DSE failures, 11 first fail at the build gate, 16 at the primary behavior gate, and 3 at the extended behavior gate. Inspection also finds missing upstream dependencies, missing version modules, unresolved imports, and source-isolation violations. These observations show that relocating source files alone does not automatically reconstruct the dependencies, package assumptions, and execution context required by an independent package boundary.

RQ2 Summary. The paired ablation shows higher functional success with repository evidence. In the separate diagnostic comparison, entryptpoint-guided mechanical relocation succeeds on 10 of 40 tasks despite explicit source locations and near-complete source overlap. These findings motivate RQ3, which analyzes confirmed source exposure and recurring implementation mismatches observed in failures after repository access.

4.3 RQ3: What Implementation Mismatches Are Observed in Source-Exposed Behavioral Failures?

We first locate the observed boundary at which each unsuccessful run ceases to satisfy the evaluation protocol. We classify each run as Pass, no usable submission, or by its first failed evaluator gate in the sequence Build–Primary–Extended–Isolation. This classification identifies where failure becomes observable; it does not by itself establish the underlying cause.

Figure 5 shows a clear pattern among the strongest configurations. Pro and Flash produce usable submissions for all 150 tasks and have no Build-first failures. Of their 77 combined unsuccessful

runs, 76 first fail at Primary or Extended, and only one first fails at Isolation. Thus, once these configurations deliver a package, the dominant remaining boundary is Primary/Extended evaluation rather than package production or loading.

Weaker configurations exhibit a broader mixture of failure modes. GLM and Qwen produce no usable submission for 38 and 25 tasks, respectively, while OSS usually delivers an artifact but has 18 Build-first and 93 Primary/Extended-first failures. Low overall success therefore need not reflect the same operational bottleneck across configurations.

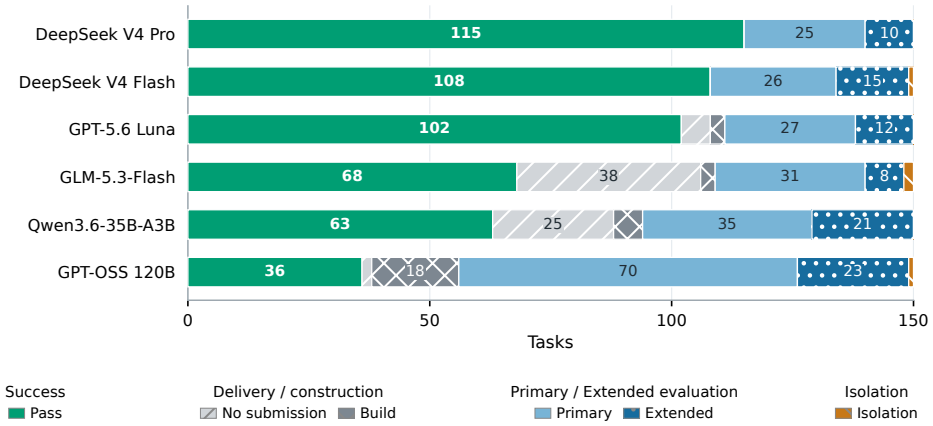


Fig. 5. Observed outcomes across 150 tasks per configuration, classified as Pass, no usable submission, or by the first failed evaluation gate in Build–Primary–Extended–Isolation order.

Primary/Extended-first failures often persist after source exposure. A Primary/Extended-first failure could arise simply because the agent never inspected implementation evidence associated with the requested capability. The saved trajectories show that this explanation is insufficient for many runs. Among the 303 Primary/Extended-first failures across all six configurations, 241 (79.5%) contain confirmed reads of endpoint-associated source-file content (Table 5). The median first confirmed read among these runs occurs at tool action five. For Pro and Flash alone, 63 of their 76 Primary/Extended-first failures contain such exposure.

These observations show that opening endpoint-associated source files is not sufficient for successful feature lifting. A confirmed read does not establish that the agent identified all supporting dependencies or understood all relevant behavioral obligations, so the result should not be interpreted as a causal diagnosis of why individual runs fail. It nevertheless rules out a simple account in which most Primary/Extended-first failures occur before any associated source content is observed.

Recurring themes across the reviewed sample. Across the 40 reviewed cases, we identified three recurring qualitative patterns: *changed ambient assumptions*, *incompatible supporting representations and pipelines*, and *destination-specific obligations requiring adaptation*. Platform-dependent path helpers in `platformdirs` and `jupyter_core`, and a construction-time scheduling bound in `apscheduler`, illustrate the first theme. Grouped diagnostics in `cerberus` and normalization before hashing in `requests_cache` illustrate the second. Resource containment in `importlib_resources`, lock-file cleanup in `filelock`, short-circuit execution in `json_logic`, and configuration parsing in `python_decouple` illustrate the third.

Table 5. Confirmed reads of endpoint-associated source-file content across 900 main-comparison outcomes. Read fractions use all runs in each row; first-read action-step medians use only confirmed reads.

Outcome	Runs	Confirmed read n/N (%)	Median first read step
Pass	492	363/492 (73.8)	5
Primary/Extended-first	303	241/303 (79.5)	5
Delivery/build	101	51/101 (50.5)	5
Isolation-first	4	4/4 (100.0)	10.5

The themes do not exhaust the sample: other cases involve local behavioral differences, missing members, an artifact constraint, or insufficient evidence for one of these relationships. We retain them as other or case-specific observations rather than forcing a theme assignment. The mapping in the replication materials records every case and reports counts only within this purposive sample. Figure 6 illustrates one instance of each recurring theme with a paired failed/successful implementation. Each pair shares the same archived public contract and pinned source; the successful counterpart passes all four functional gates.

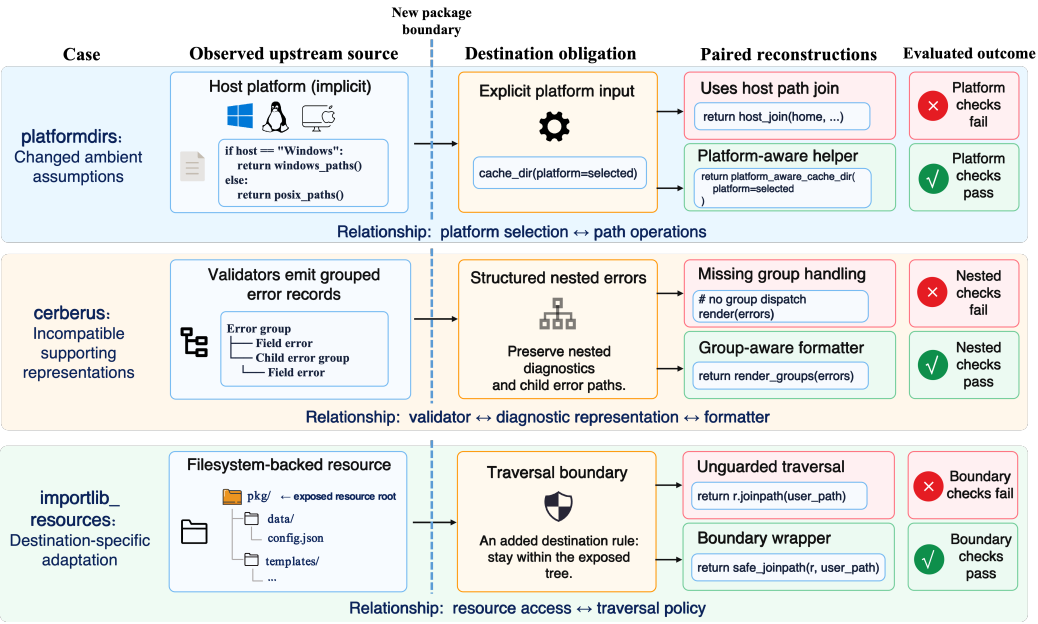


Fig. 6. Illustrative supporting relationships in Primary/Extended-first failures with confirmed source-file content reads. Each row connects upstream implementation evidence to a destination obligation and contrasts failed and passing agent artifacts under the same public contract. Code and structures are schematic; the paired artifacts may differ in multiple implementation choices.

An explicit input can invalidate an ambient assumption. In `platformdirs`, the destination contract turns platform selection into an explicit input. The failed artifact selects a Windows branch but still uses the executing platform’s path-joining operations. The successful counterpart uses a Windows-aware helper for those paths. Selecting the requested branch is therefore not enough: the helpers

below that branch must honor the same platform choice. This case illustrates adaptation of an upstream assumption rather than simply omitting an upstream API.

A preserved producer needs a compatible consumer. For `cerberus`, nested validation and diagnostic rendering share a grouped error representation. The failed artifact produces child and group records, but its formatter lacks compatible group-aware handling. The successful artifact connects group production to recursive rendering. The public obligation concerns structured nested diagnostics, so retaining validation logic alone does not ensure that the observable result is preserved. Here the relationship supports an inherited upstream behavior.

A destination restriction may require an additional adapter. For `importlib_resources`, the destination contract requires traversal to remain within an exposed resource tree. The failed artifact delegates access to an unguarded resource object; the successful counterpart routes access through a wrapper that resolves child names within the exposed tree. Containment is a destination requirement, not an upstream guarantee lost during copying. Passing the recorded traversal checks does not establish general filesystem security.

Together, these comparisons illustrate how source-backed components can be present while their assumptions or interactions remain incompatible with destination behavior. They motivate checking the relationships that realize a public obligation and identifying where adaptation changes the assumptions under which source code is reused.

4.4 Execution after an observed passing checkpoint.

We examine execution between the first reconstructed passing checkpoint and termination. Passing the frozen evaluator establishes neither complete upstream equivalence nor a stopping signal available to the agent. Outcome-level token distributions for 764 recoverable ledgers are provided in the replication materials.

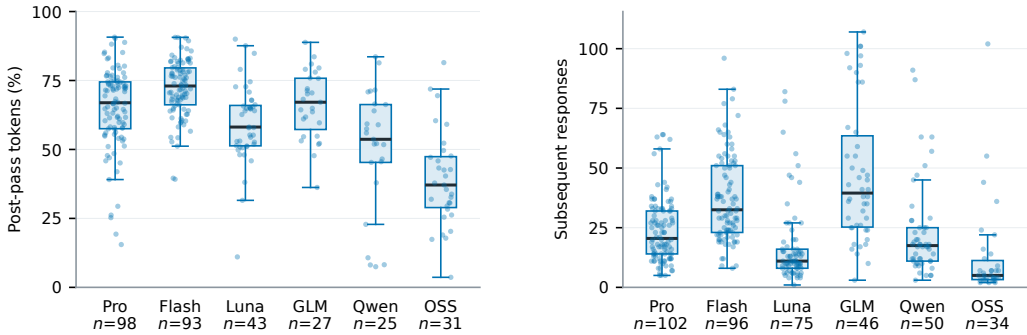


Fig. 7. Execution after the first reconstructed passing checkpoint among recoverable, finally successful runs: (a) fraction of input-plus-output tokens, including cached input (317 runs); (b) subsequent main-agent responses (403 runs). Panels use separate available samples, with configuration counts shown below the axes. Boxes show medians and quartiles, whiskers extend to 1.5 IQR, and points denote runs.

Execution continues after an observed passing artifact. Across configurations, median post-checkpoint token fractions range from 37.1% to 73.0%, and median subsequent responses range from 5 to 39.5 (Figure 7). The panels use separate recoverable samples of 317 and 403 successful runs; differing task subsets preclude a task-adjusted efficiency ranking. Continuation persists when

the checkpoint must remain passing thereafter; configuration-level medians are provided in the replication materials.

Continuation can include testing, dependency inspection, and cleanup, as in an inspected Pro JSONPath trajectory. The observed continuation motivates studying verification and stopping decisions; it does not directly quantify avoidable tokens, cost, or computation.

4.5 Implementation footprints of successful artifacts.

Functional success does not determine how a capability is reconstructed. We therefore examine two complementary properties of successful lifts: their reference-relative implementation size and their direct overlap with the source repository.

Figure 8 applies Equation 4 to 485 successful artifacts from 115 tasks and 97 repositories. Seven singleton successes are excluded.

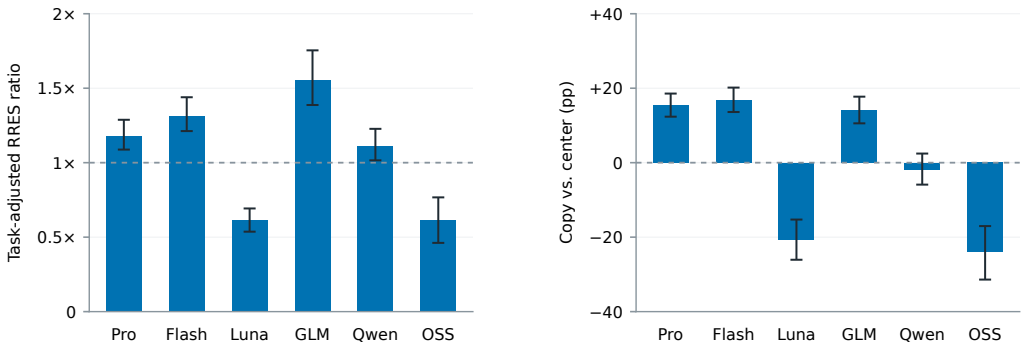


Fig. 8. Task-adjusted footprints of 485 successful artifacts across 115 tasks: (a) RRES ratios relative to the configuration-effect center; (b) Copy differences from that center in percentage points (negative values mean below-center overlap, not negative copying). Dashed lines mark the centers (1x and 0 pp); intervals are pointwise 95% task-cluster bootstrap intervals. Included successes for Pro, Flash, Luna, GLM, Qwen, and OSS are 113, 107, 99, 68, 63, and 35, respectively.

Luna and OSS are below this center on both RRES and Copy, while Pro, Flash, and GLM are above it. Qwen has an RRES ratio above one, but its Copy interval spans zero. These are footprint profiles, not a complete pairwise ranking. The replication materials provide exact estimates and sample counts; the accompanying weighting and repository-cluster checks preserve the qualitative profiles. The near-equal RRES estimates for Luna and OSS do not establish their ordering.

These success-conditioned profiles do not establish minimality, maintainability, semantic superiority, or deliberate reuse strategy, and do not extrapolate to tasks on which a configuration fails.

5 Discussion

Define reusable features by destination behavior. The paired ablation shows that repository evidence helps reconstruction, while the source-exposure analysis shows that reading associated source content does not ensure success. Together, these observations support specifying a reuse boundary through observable obligations: an entrypoint identifies where to begin inspection, but the destination contract determines what the resulting package must provide. For extraction and migration tools, this makes the contract a basis for judging whether the reconstructed capability is

sufficient, while leaving the implementation strategy open. Artifact size and source overlap can describe that implementation, but cannot substitute for behavioral verification.

Verify supporting relationships under changed assumptions. The reviewed cases illustrate how an unmet obligation can involve supporting relationships whose assumptions or representations no longer agree with the destination behavior. This motivates checking whether the components that jointly realize an obligation remain compatible across the new boundary, and whether declared adaptations require changes beyond the exported interface. Inherited behavior and destination-specific requirements need separate justification: an upstream implementation provides evidence for the former, but may require additional adaptation to satisfy the latter.

6 Threats to Validity

Scope and sampling. The 150 tasks come from 126 Python repositories and cannot represent all feature-lifting settings, languages, or dependency systems. The main comparison has one selected outcome per model-task cell, so its pass rates do not measure run-to-run variability. The paired ablation uses a stratified 40-task subset; its estimated effects need not hold across all 150 tasks. DSE is a diagnostic procedure with different inputs and operations from the agents, so its contrast with Full Source runs does not isolate a causal mechanism.

Behavioral and process measurement. Protected tests exercise stated obligations but cannot prove complete behavioral equivalence. Successful reference replays establish feasibility, not semantic completeness. The original candidate-selection funnel and a complete per-task human semantic-review ledger were not located; the present structural recheck cannot replace them. Confirmed source reads are limited to matched entrypoint-associated files and do not establish that an agent understood or used the returned code. The qualitative 40-case sample was purposively selected to examine mechanisms; its theme counts are not population prevalence estimates.

Resource and artifact measurements. The Luna and GLM token quantiles use author-confirmed aggregate totals, but complete corrected per-run provider-usage records were unavailable for independent recomputation in the retained local evidence. Retrospective checkpoint analyses use separate recoverable subsets of finally successful runs, and footprint comparisons condition on success. Their differences do not establish avoidable execution cost or a global implementation-quality ranking.

7 Related Work

7.1 Repository-Level Coding and Feature Development

SWE-bench evaluates repository patches that resolve real GitHub issues [Jimenez et al. 2024], and SWE-Bench Pro extends this direction to longer-horizon software engineering tasks [Deng et al. 2025]. Feature-development benchmarks broaden the objective beyond issue resolution. FeatBench evaluates implementation from natural-language requirements without code hints, including whether changes preserve existing features [Chen et al. 2026a]. FeatureBench derives tasks through test-driven dependency tracing and evaluates both incremental development in an incomplete repository and construction of the requested functionality from scratch [Zhou et al. 2026].

Recent benchmarks extend evaluation toward standalone repository construction. NL2RepoBench asks agents to construct an installable Python library from a natural-language requirements document and an empty workspace [Ding et al. 2026]. RepoZero formulates repository generation

as black-box reproduction from API specifications without access to the original library implementation [Zhang et al. 2026]. These settings study construction from specifications when the source implementation is unavailable.

As summarized in Table 1, these benchmarks differ from FEATURELIFTBENCH in the available construction evidence, the destination boundary and the evaluation setting.

7.2 Feature Location, Extraction, and Software Reuse

Program slicing uses dependence analysis to retain statements relevant to a specified criterion [Weiser 1984], while feature location identifies source entities associated with a requested concern [Dit et al. 2013]. Research on multi-dimensional separation of concerns addresses functionality that cuts across module boundaries [Tarr et al. 1999]. These foundations explain why recovering a reusable capability can require reasoning beyond its entrapment: the implementing statements and their supporting dependencies need not form an existing module.

Software transplantation turns this recovery problem into executable reuse. Barr et al. combine analysis, testing, and search to extract donor functionality and adapt it to a supplied host [Barr et al. 2015a]. Foundry extends transplantation to software product lines; its prodScalpel tool uses tests and annotations identifying features and implantation points to extract and integrate functionality across codebases [de Souza et al. 2025]. FEATURELIFTBENCH complements these techniques by evaluating end-to-end reconstruction into an independent package, leaving localization and implementation strategy to a general-purpose coding agent.

7.3 Behavior-Preserving Refactoring and Transformation

Classic refactoring work defines restructuring operations with preconditions under which program behavior is preserved [Opdyke 1992]. EM-Assist combines LLM suggestions with static analysis to select Extract Method candidates and uses IDE machinery to perform the transformation [Pomian et al. 2024]. SWE-Refactor evaluates atomic and compound refactorings mined from real Java projects. Its verification combines project compilation and tests with checks that the requested structural transformation occurred [Xu et al. 2026].

More recently, SWE Refactor Bench studies whole-repository stack migrations and separates migration completeness from behavioral correctness: its evaluation verifies that the requested migration occurred in addition to testing preserved behavior [Hong et al. 2026].

Feature lifting requires destination-contract behavior without prescribing a structural transformation. Accordingly, artifact size and source overlap characterize successful implementations but do not determine acceptance.

7.4 Repository Context and Evidence Utilization

RepoCoder uses iterative retrieval and generation for repository-level code completion [Zhang et al. 2023]. RepoBench evaluates retrieval, completion, and their combined pipeline [Liu et al. 2024], while CrossCodeEval targets completion requiring cross-file context [Ding et al. 2023]. These works establish repository information as a resource for generation. BeyondSWE extends evaluation to tasks requiring evidence beyond the target repository. Its SearchSWE experiments report limited and uneven gains from search augmentation and illustrate failures to integrate retrieved information with local implementation and version constraints [Chen et al. 2026b]. FEATURELIFTBENCH examines this evidence-utilization problem across a new package boundary through paired repository-evidence ablation and inspection of source-exposed failures.

8 Conclusion

We introduced FEATURELIFTBENCH, a benchmark for behavior-preserving feature lifting from real repositories. Feature lifting defines a distinct software-reuse setting in which agents must reconstruct an existing capability across a new package boundary while preserving specified behavior without runtime dependence on the original repository. FEATURELIFTBENCH provides an executable evaluation framework for this setting through behavioral contracts, protected tests, and source-free evaluation. Our study further shows that successful lifting requires more than locating or copying source code: agents must reconstruct and adapt the supporting relationships that realize behavior across the new boundary. Together, these contributions establish feature lifting as a concrete benchmark setting for studying software reuse by coding agents.

Data Availability

An anonymized replication package is being prepared. It will contain the benchmark specifications, evaluation harness, analysis scripts, saved aggregate results, and available run-level artifacts. Its access link will be added before submission.

References

- Earl T. Barr, Mark Harman, Yue Jia, Alexandru Marginean, and Justyna Petke. 2015a. Automated Software Transplantation. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis*. Association for Computing Machinery, 257–269. doi:10.1145/2771783.2771796
- Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015b. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering* 41, 5 (2015), 507–525. <https://philcmminn.com/publications/barr2015.pdf>
- Guoxin Chen, Fanzhe Meng, Jiale Zhao, Minghao Li, Daixuan Cheng, Huatong Song, Jie Chen, Yuzhi Lin, Hui Chen, Xin Zhao, Ruihua Song, Chang Liu, Cheng Chen, Kai Jia, and Ji-Rong Wen. 2026b. BeyondSWE: Can Current Code Agent Survive Beyond Single-Repo Bug Fixing? arXiv:2603.03194 [cs.CL] <https://arxiv.org/abs/2603.03194v2> Version 2, revised May 26, 2026.
- Haorui Chen, Chengze Li, and Jia Li. 2026a. FeatBench: Towards More Realistic Evaluation of Feature-level Code Generation. arXiv:2509.22237 [cs.CL] <https://arxiv.org/abs/2509.22237v2> Version 2, revised February 18, 2026.
- Leandro Oliveira de Souza, Eduardo Santana de Almeida, Paulo Anselmo da Mota Silveira Neto, Earl T. Barr, and Justyna Petke. 2025. Software Product Line Engineering via Software Transplantation. *ACM Transactions on Software Engineering and Methodology* (2025). doi:10.1145/3695987
- Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. 2025. SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks? arXiv:2509.16941 [cs.SE] <https://arxiv.org/abs/2509.16941v2> Version 2.
- Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, Xiang Gao, Chao He, Yue Hou, Fei Hu, Zhaojian Li, Weiran Shi, Zaiyuan Wang, Daoguang Zan, Chenchen Zhang, Xiaoxu Zhang, Qizhi Chen, Xianfu Cheng, Bo Deng, Qingshui Gu, Kai Hua, Juntao Lin, Pai Liu, Mingchen Li, Xuanguang Pan, Zifan Peng, Yujia Qin, Yong Shan, Zhewen Tan, Weihao Xie, Zihan Wang, Yishuo Yuan, Jiayu Zhang, Enduo Zhao, Yunfei Zhao, He Zhu, Liya Zhu, Chenyang Zou, Ming Ding, Jianpeng Jiao, Jiaheng Liu, Minghao Liu, Qian Liu, Chongyang Tao, Jian Yang, Tong Yang, Zhaoxiang Zhang, Xinjie Chen, Wenhao Huang, and Ge Zhang. 2026. NL2Repo-Bench: Towards Long-Horizon Repository Generation Evaluation of Coding Agents. arXiv:2512.12730 [cs.CL] <https://arxiv.org/abs/2512.12730v2> Version 2, revised January 8, 2026.
- Yanguibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. 2023. CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion. In *Advances in Neural Information Processing Systems*, Vol. 36. doi:10.52202/075280-2023
- Bogdan Dit, Meghan Revelle, Malcom Gethers, and Denys Poshyvanyk. 2013. Feature Location in Source Code: A Taxonomy and Survey. *Journal of Software: Evolution and Process* 25, 1 (2013), 53–95. doi:10.1002/smr.567
- Morten W. Fagerland, Stian Lydersen, and Petter Laake. 2013. The McNemar Test for Binary Matched-Pairs Data: Mid-p and Asymptotic Are Better than Exact Conditional. *BMC Medical Research Methodology* 13 (2013), 91. doi:10.1186/1471-2288-13-91

- C. A. Field and A. H. Welsh. 2007. Bootstrapping Clustered Data. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 69, 3 (2007), 369–390. doi:10.1111/j.1467-9868.2007.00593.x
- Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6 (1979), 65–70. <https://www.jstor.org/stable/4615733>
- Deyao Hong, Yizhe Chi, Wenyi Li, Xiaoqi Wang, Mingju Gao, Kaisen Yang, Bingxiang He, Youjie Zheng, Calvin Xiao, and Qinhuai Na. 2026. SWE Refactor Bench: Can Coding Agents Complete a Long-Horizon, Whole-Repository Stack Migration? arXiv:2608.23564 [cs.CL] <https://arxiv.org/abs/2608.23564v1>
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQM66>
- Charles W. Krueger. 1992. Software Reuse. *Comput. Surveys* 24, 2 (1992), 131–183. doi:10.1145/130844.130856
- Tianyang Liu, Canwen Xu, and Julian McAuley. 2024. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. In *The Twelfth International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/d191ba4c8923ed8fd8935b7c98658b5f-Abstract-Conference.html
- Bertrand Meyer. 1992. Applying “Design by Contract”. *Computer* 25, 10 (1992), 40–51. <https://se.inf.ethz.ch/~meyer/publications/computer/contract.pdf>
- William F. Opdyke. 1992. *Refactoring Object-Oriented Frameworks*. Ph.D. Dissertation. University of Illinois at Urbana-Champaign. <https://hdl.handle.net/2142/72072>
- Dorin Pomian, Abhiram Bellur, Malinda Dilhara, Zarina Kurbatova, Egor Bogomolov, Timofey Bryksin, and Danny Dig. 2024. Next-Generation Refactoring: Combining LLM Insights and IDE Capabilities for Extract Method. In *2024 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 275–287. doi:10.1109/ICSME58944.2024.00034
- Chanchal K. Roy, James R. Cordy, and Rainer Koschke. 2009. Comparison and Evaluation of Code Clone Detection Techniques and Tools: A Qualitative Approach. *Science of Computer Programming* 74, 7 (2009), 470–495. doi:10.1016/j.scico.2009.02.007
- Peri Tarr, Harold Ossher, William Harrison, and Stanley M. Sutton, Jr. 1999. N Degrees of Separation: Multi-Dimensional Separation of Concerns. In *Proceedings of the 21st International Conference on Software Engineering*. 107–119. doi:10.1145/302405.302457
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=OJd3ayDDoF>
- Mark Weiser. 1984. Program Slicing. *IEEE Transactions on Software Engineering* SE-10, 4 (1984), 352–357. doi:10.1109/TSE.1984.5010248
- Yisen Xu, Jinqiu Yang, and Tse-Hsun (Peter) Chen. 2026. SWE-Refactor: A Repository-Level Benchmark for Real-World LLM-Based Code Refactoring. arXiv:2602.03712 [cs.SE] <https://arxiv.org/abs/2602.03712v1>
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*, Vol. 37. doi:10.52202/079017-1601
- Yilun Yao, Xinyu Tan, Chao-Hsuan Liu, Yaoming Li, Zhengyang Wang, Wenhan Yu, Zhewen Tan, Yuxuan Tian, Guangxiang Zhao, Lin Sun, Xiangzheng Zhang, and Tong Yang. 2026. Harness-Bench: Measuring Harness Effects across Models in Realistic Agent Workflows. arXiv:2605.27922 [cs.AI] <https://arxiv.org/abs/2605.27922>
- Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2471–2484. doi:10.18653/v1/2023.emnlp-main.151
- Zhaoxi Zhang, Yiming Xu, Weikang Li, Jiahui Liang, and Yunfang Wu. 2026. RepoZero: Can LLMs Generate a Code Repository from Scratch? arXiv:2605.07122 [cs.SE] <https://arxiv.org/abs/2605.07122v1> Version 1, May 8, 2026.
- Qixing Zhou, Jiacheng Zhang, Haiyang Wang, Rui Hao, Jiahe Wang, Minghao Han, Yuxue Yang, Shuzhe Wu, Feiyang Pan, Lue Fan, Dandan Tu, and Zhaoxiang Zhang. 2026. FeatureBench: Benchmarking Agentic Coding for Complex Feature Development. In *The Fourteenth International Conference on Learning Representations*. <https://arxiv.org/abs/2602.10975>